# Chess_Coach

## Performance Report

Accuracy, latency and memory of the Stockfish analysis and the AI coach

Ayham Abu-Alhaj · Final project, Data Science & AI · October 2026

## 1. What was measured

Chess_Coach analyses uploaded chess games with the Stockfish engine, labels every move (Inaccuracy, Mistake, Blunder and Chess.com-style move types) and lets an AI agent built on Claude turn the mistakes into a coaching report. This report measures **the move analysis**, which every other feature depends on (sections 3–4), and **the AI coach**: its cost, latency, reliability and the quality of its advice (section 5).

- **Accuracy:** how well our Inaccuracy / Mistake / Blunder labels agree with the labels Lichess computes on its own servers.
- **Latency:** time per game and per move.
- **Memory:** peak memory of our Python code and of the Stockfish engine process.
- **Two search depths (12 and 18)**, to show the trade-off between speed and accuracy.
- **AI coach:** cost, tokens and latency per report, guardrail rejections, grounding of the cited moves and a quality rubric, over 5 runs on the same games.

Each **test case is one game**. The test set is 10 rated games from Lichess that were analysed by Lichess's servers, exported with that analysis (evaluation/data/games.pgn): **924 moves** and **118 reference labels** in total.

## 2. Test setup

| Item | Value |
|---|---|
| **Computer** | AMD Ryzen 5 5600 (6 cores / 12 threads), 23 GiB RAM, Fedora Linux |
| **Software** | Python 3.14.8, Django 6.1.2, python-chess 1.11.2, Stockfish 18 |
| **Engine settings** | 1 thread (reproducible results), 64 MB hash, fixed depth 12 and 18 |
| **Command** | python manage.py evaluate evaluation/data/games.pgn --depths 12 18 --great-overhead --repeat 3 |
| **Latency method** | time.perf_counter; each game timed 3 times, the median is reported; one warm-up engine start before measuring |
| **Memory method** | Python: tracemalloc peak, in a separate run (it slows Python by ~15%). Stockfish: peak resident memory (VmHWM in /proc/<pid>/status) |
| **Reference** | Lichess's own labels in the PGN (?! = inaccuracy, ? = mistake, ?? = blunder) |

*Table 1 – Hardware, software and measurement method.*

The Stockfish measurements were made with Python 3.14.8. The AI coach needs Python 3.12–3.13 (CrewAI does not support 3.14), so the project now runs on Python 3.13 and the coach in section 5 was measured with it.

The analysis code measured is exactly the code the web app uses (analysis_service.evaluate_moves), so the numbers describe the real application.

# 3. Accuracy

## 3.1 Metric

Every move is one prediction: "is this move an inaccuracy, mistake or blunder?". Lichess's label is the reference, ours is the prediction. A **true positive (TP)** is a move both flagged, a **false positive (FP)** a move only we flagged, a **false negative (FN)** a move only Lichess flagged.

- **Precision** = TP / (TP + FP): of the moves we flag, how many Lichess also flags.
- **Recall** = TP / (TP + FN): of Lichess's flagged moves, how many we find.
- **F1** = 2 · Precision · Recall / (Precision + Recall): one number that balances both.
- **Same severity:** of the moves both flagged, how often the label is identical.

Totals are micro-averaged: TP, FP and FN are summed over all 10 games first, so long games count more than short ones.

## 3.2 Results

| Depth | Precision | Recall | F1 | Lichess labels | Our labels | Both (TP) | Same severity |
|---|---|---|---|---|---|---|---|
| 12 | 0.667 | 0.712 | 0.689 | 118 | 126 | 84 | 57/84 (68%) |
| 18 | 0.773 | 0.78 | 0.776 | 118 | 119 | 92 | 69/92 (75%) |

*Table 2 – Accuracy against Lichess, all 10 games.*

Going from depth 12 to 18 raises F1 from **0.689 to 0.776**, precision from 0.667 to 0.773 and recall from 0.712 to 0.78. The severity also agrees more often (68% → 75%).

## 3.3 Where we disagree – confusion matrices

Rows are Lichess's label, columns ours. The shaded diagonal is agreement.

| | Ours: none | Ours: inaccuracy | Ours: mistake | Ours: blunder |
|---|---|---|---|---|
| **Lichess: none** | 764 | 38 | 4 | 0 |
| **Lichess: inaccuracy** | 24 | 28 | 3 | 0 |
| **Lichess: mistake** | 10 | 13 | 6 | 3 |
| **Lichess: blunder** | 0 | 3 | 5 | 23 |

*Table 3 – Confusion matrix, depth 12.*

| | Ours: none | Ours: inaccuracy | Ours: mistake | Ours: blunder |
|---|---|---|---|---|
| **Lichess: none** | 779 | 25 | 2 | 0 |
| **Lichess: inaccuracy** | 23 | 27 | 5 | 0 |
| **Lichess: mistake** | 2 | 10 | 17 | 3 |
| **Lichess: blunder** | 1 | 0 | 5 | 25 |

*Table 4 – Confusion matrix, depth 18.*

- **Blunders are found reliably.** Of Lichess's 31 blunders, depth 12 flagged 31 in some way (23 as blunder); depth 18 flagged 30 (25 as blunder).
- **Most disagreements are borderline inaccuracies.** At depth 12, 38 moves were an inaccuracy only for us and 24 only for Lichess. An inaccuracy starts at a 10% drop in winning chances; a move near that line flips with a small difference in evaluation.

- **Mistake vs inaccuracy is the hardest boundary**, for the same reason: several Lichess mistakes were inaccuracies for us.

## 4. Latency and memory per test case

Depth 12 – one row per game (test case):

| Game | Moves | Time (s) | ms / move | Python peak (MB) | Stockfish peak (MB) | Great check +% | Precision | Recall | F1 |
|---|---|---|---|---|---|---|---|---|---|
| zR8HRccT | 127 | 4.27 | 34 | 1.10 | 750.7 | 30.1 | 0.714 | 0.577 | 0.638 |
| HmLzqg5C | 118 | 3.79 | 32 | 1.06 | 750.6 | 29.1 | 0.500 | 0.583 | 0.538 |
| kAdOQKeh | 136 | 6.14 | 45 | 1.18 | 750.8 | 43.7 | 0.722 | 0.929 | 0.813 |
| xKWdG1d1 | 89 | 4.76 | 53 | 0.92 | 751.0 | 51.4 | 0.882 | 0.882 | 0.882 |
| SnXhGC57 | 114 | 4.68 | 41 | 1.05 | 750.9 | 52.3 | 0.444 | 0.667 | 0.533 |
| nrmBGiQF | 49 | 2.73 | 56 | 0.79 | 750.7 | 32.2 | 0.714 | 0.833 | 0.769 |
| VOqkxbKK | 56 | 3.48 | 62 | 0.82 | 751.1 | 41.1 | 0.714 | 0.556 | 0.625 |
| haRbNEM1 | 53 | 3.27 | 62 | 0.80 | 750.9 | 39.9 | 0.167 | 1.000 | 0.286 |
| SNG5EiHV | 54 | 3.20 | 59 | 0.89 | 750.4 | 39.8 | 1.000 | 0.667 | 0.800 |
| ItgC9JQz | 128 | 5.58 | 44 | 1.02 | 750.5 | 52.3 | 0.619 | 0.722 | 0.667 |

*Table 5 – Latency, memory and accuracy per game, depth 12.*

Depth 18:

| Game | Moves | Time (s) | ms / move | Python peak (MB) | Stockfish peak (MB) | Great check +% | Precision | Recall | F1 |
|---|---|---|---|---|---|---|---|---|---|
| zR8HRccT | 127 | 80.52 | 634 | 1.30 | 750.7 | 15.8 | 0.720 | 0.692 | 0.706 |
| HmLzqg5C | 118 | 86.33 | 732 | 1.28 | 750.3 | 11.5 | 0.579 | 0.917 | 0.710 |
| kAdOQKeh | 136 | 96.20 | 707 | 1.39 | 750.8 | 31.7 | 0.857 | 0.857 | 0.857 |
| xKWdG1d1 | 89 | 72.25 | 812 | 1.25 | 750.5 | 40.5 | 0.778 | 0.824 | 0.800 |
| SnXhGC57 | 114 | 66.58 | 584 | 1.20 | 750.9 | 20.5 | 1.000 | 0.667 | 0.800 |
| nrmBGiQF | 49 | 34.54 | 705 | 0.92 | 750.8 | 26.2 | 0.667 | 1.000 | 0.800 |
| VOqkxbKK | 56 | 47.92 | 856 | 1.04 | 749.9 | 34.1 | 0.800 | 0.889 | 0.842 |
| haRbNEM1 | 53 | 36.22 | 683 | 1.14 | 751.1 | 17.5 | 1.000 | 1.000 | 1.000 |
| SNG5EiHV | 54 | 47.25 | 875 | 1.09 | 750.3 | 27.1 | 1.000 | 0.778 | 0.875 |
| ItgC9JQz | 128 | 74.64 | 583 | 1.27 | 750.8 | 27.5 | 0.917 | 0.611 | 0.733 |

*Table 6 – Latency, memory and accuracy per game, depth 18.*

| Depth | Total time | Per game | Per move | Python peak | Stockfish peak | Great check cost |
|---|---|---|---|---|---|---|
| 12 | 41.9 s | 4.19 s | 48.8 ms | 1.18 MB | 751.1 MB | +41.19% |
| 18 | 642.5 s | 64.25 s | 717.1 ms | 1.39 MB | 751.1 MB | +25.24% |

*Table 7 – Latency and memory summary.*

- **Depth 18 is 14.7× slower per move** (48.8 ms → 717.1 ms). A typical game takes 4.19 s at depth 12 and 64.25 s at depth 18.
- **Memory is dominated by Stockfish** (about 751 MB at peak) and does not depend on depth: it is the engine's neural network plus its 64 MB hash table, for two engine processes (the second one runs the Great check). Our Python code needs only about 1.18–1.39 MB.
- **The Great move check** (a second search, only when the player found the best move) costs **+41.19%** time at depth 12 and **+25.24%** at depth 18.

### 4.1 Is the system real-time?

The course asks which operations are real-time, near real-time or delayed. We classify each operation by how long the user waits and how the interface handles it:

| Operation | Measured time | Classification | How the app handles it |
|---|---|---|---|
| Pages, uploading a PGN, ticking practice items | a fraction of a second (database only, no AI) | Real-time | Normal request – response |
| Stockfish analysis, depth 12 (default) | 48.8 ms per move, 4.19 s per game | Near real-time | Background worker; the page refreshes itself when the analysis is ready |
| Stockfish analysis, depth 18 | 717.1 ms per move, 64.25 s per game | Delayed (batch) | Suitable only for offline / deep analysis |
| AI coaching report (agent) | 17.72 s on average (5 games, Sonnet; section 5) | Delayed | Background task; the report page shows progress and refreshes itself |

*Table 8 – Response time and real-time classification of each operation.*

Because the slow operations (engine analysis and the AI agent) run in a background worker (Django Tasks + django-tasks-db), the web interface itself always answers immediately: the user never waits on a frozen page, and status endpoints let the pages update themselves. The page and upload times were not timed with a script; they involve no engine or AI work.

**Differences between test cases.** Latency grows with the length of the game (49 to 136 moves) and with the number of "Great" checks, so longer, sharper games take longer. Memory is almost the same for every game (Stockfish ≈ 751 MB, Python ≈ 1 MB): it is fixed by the engine's neural network and hash table, not by the game.

## 5. The AI coach (agent)

The coach was measured with a script, so the numbers can be reproduced. **python manage.py evaluate_coach** runs the same coaching flow as the web app (same move data, prompts, tools and guardrail) several times on the same games and records every run. It does not save reports, so the evaluation does not change the user's data.

### 5.1 Setup

| Item | Value |
|---|---|
| **Model** | Claude Sonnet 5.5 (anthropic/claude-sonnet-5-5): $2 / $10 per million input / output tokens |
| **Software** | Python 3.13, CrewAI 1.15.28, on the same computer as Table 1 |
| **Test case** | One complete report on the user's 5 latest analysed games (135 of the player's own moves); the same games in every run |
| **Runs** | 5, each with a new model object (so the token counts of one run do not include the previous runs) |
| **Command** | python manage.py evaluate_coach <username> --runs 5 --games 5 |
| **Latency** | time.perf_counter around the whole flow (all model requests and tool calls) |
| **Tokens and cost** | Token usage reported by the AI provider for the run × the price per token |
| **Guardrail** | Rejections recorded by the guardrail, by reason (structure, unknown games, invented moves) |
| **Grounding** | Every move cited in the evidence is checked against the player's moves and the engine's best moves, with the same rules as the guardrail |
| **Quality** | Rubric: relevant, specific and correct, 1–5, for every weakness |

*Table 9 – How the AI coach was measured.*

## 5.2 Cost and latency

| Run | Time (s) | Model requests | Tool calls | Tokens in | Tokens out | Cost |
|---|---|---|---|---|---|---|
| 1 | 19.64 | 2 | 3 | 8,188 | 1,903 | $0.0354 |
| 2 | 18.47 | 2 | 3 | 8,188 | 2,084 | $0.0372 |
| 3 | 15.86 | 2 | 3 | 8,188 | 1,892 | $0.0353 |
| 4 | 15.89 | 2 | 3 | 8,188 | 1,826 | $0.0346 |
| 5 | 18.74 | 2 | 3 | 8,188 | 2,062 | $0.0370 |
| Mean | 17.72 | 2 | 3 | 8,188 | 1,953 | $0.0359 |

*Table 10 – Cost and latency per report, Claude Sonnet 5.5.*

- **A report costs about $0.036 with Sonnet** ($0.0346–$0.0372); all 5 runs together cost $0.18. 1,000 reports would cost about $36.
- **The input is the same in every run** (8,188 tokens): for the same games the prompt, the tool definitions and the tool results do not change. Only the written answer varies (1,826–2,084 output tokens).
- **The answer drives the cost:** output is 19% of the tokens but 54% of the price, because an output token costs 5× more than an input token.
- **Latency:** 17.72 s on average (15.86–19.64 s) for 2 model requests and 3 tool calls. The report runs in the background worker, so the web page never waits for it.
- **Cost control:** only one report per user can be in progress, and MAX_REPORTS_PER_DAY (default 10) limits one user to about $0.36 per day with Sonnet, on the web page and on the API.

## 5.3 Cost with other models

| Model | Price in / out (per million tokens) | Cost per report | 1,000 reports |
|---|---|---|---|
| **Claude Haiku 5.5** | $0.10 / $0.50 | ≈ $0.0018 | ≈ $1.80 |
| **Claude Sonnet 5.5 (measured)** | $2 / $10 | $0.0359 | ≈ $36 |
| **Claude Opus 5.5** | $4 / $20 | ≈ $0.0718 | ≈ $72 |

*Table 11 – Cost per report for each model, estimated from the measured Sonnet tokens.*

Haiku and Opus are estimates: another model may make a different number of requests and write a shorter or longer answer. Two earlier reports made with Haiku cost $0.0016 and $0.0036 (3 and 4 model requests), in the same range as the estimate. The cost follows the number of model requests more than the number of games, because every request sends the instructions and the earlier tool results again. The model is chosen in .env (AI_MODEL), so the price can be changed without changing the code.

## 5.4 Reliability and grounding

| Measure | Result |
|---|---|
| **Runs that ended with an accepted report** | 5 / 5 |
| **Guardrail rejections (each one is a retry)** | 0 in 5 runs |
| **Weaknesses per report** | 3 in every run |
| **Cited moves that are in the player's data** | 104 / 104 (100%) |
| **Weaknesses that cite at least one move** | 15 / 15 |

*Table 12 – Reliability and grounding of the coach.*

- **100% grounding is expected:** the guardrail rejects any report that cites a game or a move that is not in the data, so an accepted report cannot contain one. With 0 rejections, Sonnet never needed a retry in these runs. The rejection path itself is covered by automated tests that use a fake model.

- **The 100% is not empty:** all 15 weaknesses cite real moves (on average 6.9 per weakness).
- **Consistent:** all 5 runs found the same three themes: pieces left hanging in the middlegame, missed checks and forcing moves, and accuracy dropping after the opening.

### 5.5 Quality of the advice (rubric)

The 15 weaknesses (5 runs × 3) were scored from 1 to 5: **relevant** (it matches where the player loses points), **specific** (concrete moves and a habit to practise, not generic advice) and **correct** (the explanation and the evidence agree with the data). **The scores were given by an AI reviewer (Claude), with a written reason for every score in docs/coach_evaluation_rubric.csv. They are not a human or expert rating, and an AI judging AI-written advice is a limitation of this measurement.**

| Criterion | Average | Lowest | Highest |
|---|---|---|---|
| **Relevant** | 4.27 | 3 | 5 |
| **Specific** | 4.40 | 3 | 5 |
| **Correct** | 4.27 | 3 | 5 |
| **All three** | 4.31 | | |

*Table 13 – Rubric scores (1 = poor, 5 = excellent).*

The review found problems that the grounding check cannot see. Every cited move was real, but some statements about those moves were not:

- **Game phase:** in run 1, the same two moves (19. Nge4 and 41... Rf4) are called middlegame blunders in one weakness and endgame examples in another.
- **Counting:** "16 mistakes and 10 blunders" means 16 errors, 10 of them blunders, but reads as 26.
- **Overstatement:** "you often miss checks" for 3 cases in 135 moves.
- **Repetition:** in 3 runs the third weakness repeats the first one in other words.

## 6. Analysis

1. **Speed vs accuracy.** Depth 18 improves F1 by 0.087 but costs 14.7× more time. For an interactive web app, depth 12 (about 4 s per game) is the better default; depth 18 is better for offline, "deep" analysis.
2. **The reference is not perfect truth.** Lichess analyses on its own servers with a different Stockfish setup (version, depth, time). Even a perfect copy of its rules cannot match it 100%; part of the remaining disagreement is noise in the reference.
3. **Errors sit at thresholds.** Almost all disagreements are between neighbouring labels (none ↔ inaccuracy, inaccuracy ↔ mistake). The labels that matter most for coaching – blunders – are nearly always found.
4. **Small games make unstable scores.** Game haRbNEM1 had a single Lichess label: at depth 12 our 5 extra labels gave it F1 0.286, at depth 18 it reached 1.000. Totals over all games are therefore more meaningful than single-game scores.
5. **Horizon effect.** An earlier test on the Opera Game showed the same pattern: at depth 12, 15.Bxd7+ was wrongly called a blunder because the engine did not see far enough; at depth 20 the label disappeared.
6. **Memory is not a problem for Python, but for the engine.** Our code uses about 1 MB; Stockfish uses hundreds of MB per process.
7. **The coach is cheap and grounded, but not flawless.** At about $0.036 and 18 s per report with Sonnet it is practical for a web app. The guardrail guarantees that every cited move is real, but the rubric shows that the text around the moves can still be wrong (game phase, counts, "often").

# 7. Improvements

## 7.1 Already made during the project

- **Warm-up and median of 3 runs** – the first engine start of a session was ~30% slower because the neural network loads from disk; timing now starts after a warm-up and reports the median.
- **Memory measured in a separate run** – tracemalloc slowed Python by ~15% and distorted the timings.
- **Great check only when it can matter** – only for the engine's best move, never for obvious moves (out of check, recaptures, free pieces), not in the first opening moves and not in decided positions.
- **Separate engine for the Great check** – running the extra search on the main engine changed its hash table and slightly changed the labels of later moves; with a second process the evaluations are identical with and without the check.
- **Lichess thresholds kept for mistakes** – so our labels stay comparable with Lichess, while the Chess.com-style types are added on top.
- **Evidence guardrail** – the guardrail now checks every move cited as evidence (move number, side and move) against the player's data, not only the game ids.
- **Correct token counting** – the first coach evaluation used one model object for all runs; CrewAI keeps a running token total on it, so runs 2–5 showed the cost of all earlier runs as well ($0.035, $0.071, … $0.177). Each run now gets a new model object, and a test checks that two runs report the same tokens.
- **Daily report limit** – MAX_REPORTS_PER_DAY, checked in the service, limits the AI cost per user on both the web page and the API (HTTP 429).

## 7.2 Proposed next steps

- **A middle depth (14–16)** as default: likely most of the accuracy gain for a fraction of the time (to be measured with the same command).
- **Adaptive depth:** re-check only moves close to a label threshold at a higher depth – these are exactly the moves where we disagree with Lichess.
- **Reuse one engine per worker** across games instead of starting Stockfish for every game, saving the start-up time (loading the neural network) for each game.
- **More threads per engine** for speed (results become slightly less reproducible), or a time limit per move instead of a fixed depth.
- **A larger test set** (more players and rating levels) – 10 games and 118 labels give a first estimate, not a precise one.
- **A human expert rubric** on more reports and players, and an **AI judge** checked against those human scores, so the quality can be measured automatically.
- **Check the claims, not only the moves:** the guardrail could also check the game phase and the numbers cited in the evidence against the tool results.
- **Prompt caching** of the fixed instructions and tool definitions (the input is identical in every run) could lower the input cost; not yet measured.

# 8. How to reproduce

All numbers in sections 3 and 4 come from docs/evaluation_results.csv and docs/evaluation_results.txt, produced by:

```
python manage.py evaluate evaluation/data/games.pgn --depths 12 18 --great-overhead --repeat 3
```

Results depend on the Stockfish version, the depth and the computer; with another Stockfish version a few labels and timings will differ.

The numbers in section 5 come from docs/coach_evaluation.csv and docs/coach_evaluation.txt, produced by:

```
python manage.py evaluate_coach <username> --runs 5 --games 5
```

The same command writes docs/coach_evaluation_rubric.csv with one row per weakness and empty score columns; the rubric scores and their reasons were added to that file afterwards. The coach evaluation calls the real AI model (about $0.04 per run with Sonnet), and the numbers change a little from run to run because the model's answers vary.